

Retry Synthesis with Bounded Calls and Protected Work

ANONYMOUS AUTHOR(S)

Retry limits bound failed attempts, but protected fallback can move whole transformations into critical sections. We study how to bound both calls and protected work without knowing how many concurrent writes will occur. For dependent, persistent transformations with fixed whole-kernel work, one policy uses at most $n + r$ validation or completion calls and simultaneously minimizes worst protected work for every write budget B : the sum of the largest $\min((B - r)_+, n)$ job works. Without cached completion, a policy satisfying the same universal call cap must protect all work already at $B = r$. A potential justifies the atomic-call model despite retained preparations, and actual-write environments match the bounds. For declared scalar prices, we separately construct checkable all-budget policies and identify a class with compact compilation. A Roslyn source modification preserves checked event, snapshot and compiler projections while bounding calls. In authored arrival experiments, cached completion reduces background delay at an intermediate arrival interval but increases foreground time under bursts. A source-permitted partial-rebase comparator avoids full protected recomputation in the tested unrelated-update workload, but retains protected merge cost. A partial-repair counterexample shows why the simultaneous optimum depends on whole-kernel operations.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; *Concurrent programming structures*.

Additional Key Words and Phrases: program synthesis, concurrency, retries, quantitative games, resource guarantees

1 Introduction

A retry limit bounds unsuccessful attempts, but also creates a design choice: what should happen after the limit is reached? Recomputing under protection guarantees completion, yet moves an entire transformation into a critical section. Reusing a prepared result when it remains current can avoid that work. We ask for a guarantee that holds even when the number of concurrent writes is unavailable to the policy.

Consider two dependent transformations with works 8 and 16 and a cap of two validation or completion calls. Every call must complete a transformation. Fresh protected completion performs all 24 units under protection. Cached completion instead prepares outside and, within one completing callback, either reuses the preparation or recomputes after a mismatch. Its worst protected work is 0, 16, or 24 when at most 0, 1, or 2 writes occur (Figure 1). The policy does not receive this write budget. The benefit has a cost: at one write its worst total work is 40, compared with 24 for fresh protected completion. The choice concerns which resource bounds can be met together.

Before the retry cap		Prepare outside → validate: complete or fail			
After the cap, two modes		Fresh protected computation → complete			
After the cap, three modes		Prepare outside → protected reuse/recompute → complete			
Write bound B	Worst protected work L		Worst total work W		
	Two modes	Three modes	Two modes	Three modes	
0	24	0	24	24	
1	24	16	24	40	
2	24	24	24	48	

Fig. 1. Completing modes after a retry cap, and the two-job example with $r = 0$. Both fixed policies satisfy $Q \leq 2$ in every finite-write environment; neither receives B . Values follow from the model, not timing measurements.

The main result generalizes this example to any dependency DAG and allowance r of extra calls. It proves a lower bound over the atomic-call interface and gives an exact suffix certificate for further protection choices. A separate priced synthesis problem takes a write allowance as input and returns optimal policies across all budgets.

Fallback and quantitative choices among retries, guards and caches are established [1, 20, 21], as are budgeted DP and input-slope envelopes [10, 30, 34]. We add three specialized arguments: the universal call/protection frontier and suffix feasibility; an initial-state normal-form potential covering retained preparations with matching actual-write environments; and finite all-budget closure with compact compilation on a compatible mixed-price class. Roslyn illustrates the count tradeoff and conditional timing effects; its variable work does not meet the priced contract.

2 Charged Atomic Calls

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$ form a finite DAG with predecessor set $\text{Pred}(i)$. Readiness requires all predecessors completed. Completion records the exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Its fixed unprotected work is $w_i > 0$; under protection it costs $w_i + p_i$, $p_i \geq 0$, including predecessor processing. Kernels finish with exact results; partial/incremental evaluation, preparation before readiness and discharge by external work are excluded. A preparation is a capability tied to its input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

Local states are U (unprepared), C (current preparation), or D (completed), initially all U. Several ready jobs may be in C. An invalidation changes an own-input identity, preserving captured predecessor outputs. The environment has a global remaining allowance b , initially $B \in \mathbb{N}$. At zero allowance only matching outcomes are legal.

The declared charge $v_i \geq 0$ covers an identity-conditional publication call. The charge $k_i \geq 0$ covers entry and exit of an atomic callback, including a callback that only validates. It may be supplied as $g_i + r_i$. These charges define a resource metric; they are not measurements of API latency. All prices are integers, or rational prices scaled uniformly to integers. Table 1 defines the available counted operations. A callback holds protection only during that operation and performs no other job's foreground work.

Table 1. Counted atomic-call interface. A mismatch spends one invalidation token. Cached completion always finishes the job; validation-only mismatch retains the current live entry and reports failure.

Operation	Start	Matching outcome/cost	Mismatching outcome/cost
Prepare	U	C, w_i	—
Conditional publication	C	D, v_i	U, v_i
Validation-only callback	C	D, k_i	U, k_i
Fresh callback	U	D, $k_i + w_i + p_i$	—
Cached callback	C	D, k_i	D, $k_i + w_i + p_i$

Discarding a preparation is free. A free inspection is permitted only with a legal outcome that preserves a currently usable preparation; it cannot certify that future interference is exhausted. Programs can select any ready job after any completed call, retain preparations or repeat inspections. There is no bare acquire/release operation, guard-retaining return or cross-job batch callback. All

calls finish, and every noncompleting program, including one executing free cycles forever, has infinite objective. The environment resolves outcomes after observing the program's action.

Retained guards permit additional choices; Section 11 preserves the zero-fee distinction and charged counterexamples.

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S , let $\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ denote the ready jobs.

Let W count all foreground kernel work, L the part performed inside atomic callbacks, and Q all foreground completion/validation calls, including failed conditionals. Reads, initialization and external-writer calls are excluded from Q . Here w_i is each complete kernel's mandatory work on every input, inside or outside protection, not merely an upper bound. Let $\Omega_k(T)$ sum the largest $\min(k, |T|)$ works in T , and $\Omega(T)$ its total. Omitted sets mean J ; $\Omega_0 = 0$ and $\Omega_n = \Omega$.

THEOREM 1 (PROTECTED WORK UNDER A CALL CAP). *Under Theorem 7's interface and environment, fix integers $B, r \geq 0$. Among programs that complete all jobs with $Q \leq n + r$ on every execution, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Fresh attempts have disjoint intervals between read and comparison; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For the two-mode lower bound when $B > r$, reject each cheap call with a fresh own-key write, stopping writes after the $(r + 1)$ st rejection. This uses at most $r + 1 \leq B$ writes. An admissible program cannot incur that many noncompleting calls, so all its jobs complete under protection.

For three modes with $B > r$, target the $k = \min(B - r, n)$ largest jobs. Reject their cheap calls and force their cached calls to recompute by fresh own-key writes immediately before comparison; fresh callbacks already incur w_i inside. Stop writes after $r + 1$ target cheap rejections. Before a next write, at most r such rejections and $k - 1$ cached target completions have occurred, so that write still uses at most $r + k \leq B$ writes. Admissibility excludes $r + 1$ noncompleting calls. Thus every target completes with protected work, giving $L \geq \Omega_k$. Fresh identities invalidate retained preparations, and no inspection or completed callback retains a guard across the chosen boundary. $\square$

THEOREM 2 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in

a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the ratio is $n/\min(B - r, n)$, reaching n at $r = B - 1$. Linear prices $W + \lambda L + \kappa Q$ correspond to $p_i = \lambda w_i$, $v_i = k_i = \kappa$; these counts do not estimate throughput.

Work cost of simultaneous protection optimality. Among $\mathcal{U}_3(0)$ programs attaining the optimal worst- L curve for every budget, minimum worst W at B is $\Omega + \Omega_{\min(B, n)}$. All-cached attains it with $Q = n$, $W = \Omega + L$. To prove necessity, target the $k = \min(B, n)$ largest jobs. Each counted call must complete. Immediately before each target's first comparison/acquisition, insert a fresh write exactly if its kernel was previously executed. Every target then executes inside. If this uses $h \leq k$ writes, the simultaneous guarantee gives $\Omega_k \leq L \leq \Omega_h$. Positivity forces $h = k$: each target pays a prior and a protected kernel. Unknown B alone does not force this cost; the artifact gives a counterexample.

For general r , consider the threshold family that freshly prepares before each attempt, uses cheap validation until r global failures, and then cached completion. It may adaptively choose another ready job after any failure, but retains no other preparation.

PROPOSITION 3 (WORK-OPTIMAL CHOICE WITHIN THE THRESHOLD FAMILY). *Minimum-work ready selection minimizes worst W within this family simultaneously for every B, r , even against selectors that receive B . A heap constructs its fixed order in $O(|E| + n \log(n+1))$ operations. This is not optimality among all causal programs or the full (W, L, Q) Pareto frontier.*

PROOF. Let $X(S, b, t)$ be optimal excess work over mandatory work, with t cheap failures left. At $t = 0$ it is the sum of the largest $\min(b, |S|)$ works in S ; at $b = 0$ it is zero. For $b, t > 0$,

$$X(S, b, t) = \min_{i \in \mathcal{A}(S)} \max\{X(S - i, b, t), w_i + X(S, b - 1, t - 1)\}.$$

If $T \subseteq S$, simulating omitted jobs as successful virtual attempts shows $X(T, b, t) \leq X(S, b, t)$: they consume neither failures nor slack. For ready i, j with $w_i \leq w_j$, compare choosing j first with choosing i , then j after success. Failure of i , success of i then failure of j , and both successes are each bounded by the j -first value, by weight order and subset monotonicity. Hence i is optimal at every state. Failure changes neither readiness nor weights, so the rule repeats the current job independently of b, t . $\square$

Certifying post-threshold protection choices. After r cheap failures, let h count cached mismatches and ℓ work already protected. Consider a boundary with unfinished jobs S , no retained preparation, and an observed prefix admitting exactly $r + h$ actual writes. This holds for the modeled serial histories; it does not compress histories revealing additional writes.

THEOREM 4 (EXACT SUFFIX FEASIBILITY). *There exists a continuation with at most $|S|$ further calls that meets Theorem 2's protection bound in every finite-write extension iff*

$$C(S, h, \ell) : \quad \ell + \Omega_t(S) \leq \Omega_{h+t}(J) \quad (0 \leq t \leq |S|). \quad (1)$$

PROOF. No further rejectable call is allowed: another adversarial write could force a failure beyond the cap. For necessity, target the t largest remaining jobs. A fresh callback already protects its target; otherwise one fresh own-key write before the eventual comparison invalidates its retained preparations, forcing protected recomputation. At most $u \leq t$ writes force $L \geq \ell + \Omega_t(S)$, while the guarantee requires $L \leq \Omega_{h+u}(J) \leq \Omega_{h+t}(J)$. Inspections cannot retain protection and noncompletion is inadmissible. For sufficiency, freshly prepare/cached-complete in any legal order. Its u mismatches concern distinct jobs, giving $L \leq \ell + \Omega_u(S)$. Every call completes. $\square$

Fresh completion of ready i is therefore feasible exactly when $C(S - i, h, \ell + w_i)$ holds. Cached matching preserves C by subset monotonicity; on mismatch use $w_i + \Omega_t(S - i) \leq \Omega_{t+1}(S)$. These finite inequalities certify allowable actions without B .

The *all-tail* rule finishes all remaining jobs freshly when $\ell + \Omega(S) \leq \Omega_h(J)$; otherwise it continues cached completion. A different rule greedily takes each certified fresh next job.

PROPOSITION 5. *Either rule retains the exact worst- L frontier and universal call cap, and weakly reduces worst W relative to the same-order threshold policy.*

PROOF. At the threshold switch $C(S, 0, 0)$ holds. Certified choices preserve it, every later call completes, and Theorem 2 gives equality. For work, replace each fresh completion in a run by a threshold cached match, retaining every other outcome. This same-order execution has equal W and no additional writes. $\square$

On chain $w = (2, 4, 1, 1)$, $r = 1$, $B = 4$, all-tail reduces worst W from 18 to 17 at $L = 7$, $Q = 5$; both return to 18 at $B \geq 5$. The two certified rules are incomparable (Section 9); feasibility does not establish general W -optimality.

4 A Three-Mode Normal Form

Write S for unfinished jobs and $\mathcal{A}(S)$ for ready jobs. Reachable unfinished sets are upsets. For proofs we extend the game to all subsets, treating absent predecessors as completed. Define

$$\begin{aligned} m_i &= \min(v_i, k_i), & c_i &= w_i + m_i, \\ \delta_i &= k_i - m_i, & P_i &= p_i + \delta_i, \\ s_i &= w_i + p_i. \end{aligned} \tag{2}$$

An unavoidable baseline from an all-unprepared start is $A(S) = \sum_{i \in S} c_i$. Three serial modes remain at boundaries where unfinished jobs have no retained preparation:

- (1) *Protected*: execute a fresh callback, paying $c_i + P_i$, and remove i .
- (2) *Cheap validation*: prepare and invoke the cheaper validation API. A match pays c_i and removes i ; a mismatch pays c_i , leaves i unprepared and spends one token.
- (3) *Cached completion*: prepare and invoke a cached callback. A match pays $c_i + \delta_i$; a mismatch pays $c_i + \delta_i + s_i$ and spends one token. Both remove i .

Subtract the once-paid baseline. The optimal worst excess of these modes satisfies $F(\emptyset, b) = F(S, 0) = 0$ and, for $b > 0$,

$$F(S, b) = \min_{i \in \mathcal{A}(S)} \left\{ \begin{aligned} &P_i + F(S - i, b), \\ &\max\{F(S - i, b), c_i + F(S, b - 1)\}, \\ &\delta_i + \max\{F(S - i, b), s_i + F(S - i, b - 1)\} \end{aligned} \right\}, \tag{3}$$

where $S - i = S \setminus \{i\}$. At $b = 0$ cheap validation attains zero excess; the cached option has excess δ_i , without a mismatch branch. Each mode outcome decreases $|S| + b$, so a minimizing deterministic controller finishes within $n + B$ mode invocations. This is worst-path control against an adaptive environment, not expected-cost control against a fixed distribution.

THEOREM 6 (CHARGED ATOMIC-CALL NORMAL FORM). *For the interface of Section 2, the minimum worst cost over all causal programs from the all-unprepared state is*

$$\text{OPT}(J, B) = \sum_{i \in J} c_i + F(J, B).$$

A serial three-mode controller attains it. Programs retaining several preparations or changing jobs between calls cannot improve the value.

PROOF. The three modes implement Table 1 with exactly Equation (3)’s full costs. Their decreasing rank and captured successful outputs give the upper bound.

For the lower bound, let x be any interface state with unfinished set T . Define

$$\Phi(x, b) = \sum_{i: x_i = U} c_i + \sum_{i: x_i = C} m_i + F(T, b). \quad (4)$$

Cached jobs remain ready because completion persists. For each operation we show a legal outcome preserving immediate cost plus successor potential. Preparation pays $w_i = c_i - m_i$ and replaces c_i by m_i . Validation with charge $z_i \in \{v_i, k_i\}$ gives excess $z_i - m_i + F(T - i, b)$ on success or $w_i + z_i + F(T, b - 1)$ on failure. Since $z_i \geq m_i$, their maximum dominates the cheap branch; at $b = 0$ success suffices. Fresh callbacks use the protected inequality. Cached callbacks give excesses $\delta_i + F(T - i, b)$ and $\delta_i + s_i + F(T - i, b - 1)$, exactly the cached branch. Discarding C raises potential by w_i ; inspection admits a preserving outcome.

Nature selects a witnessing branch after each action, within budget b . Telescoping gives the initial potential as a cost lower bound, covering arbitrary job switching and retained preparations. Noncompletion costs infinity; intermediate potentials need not equal intermediate optima. $\square$

A resource cap M is feasible exactly when $A(J) + F(J, B) \leq M$. Prices and allowance are supplied; restricted or correlated outcomes preserve the upper guarantee but may lower the optimum.

5 Actual External Writes in Java

Use Java 17’s `ConcurrentHashMap` with one foreground thread, distinct persistent keys, immutable identity-equal wrappers and fresh, never-reused identities for external writes. Each job updates only its fixed key. Keys are not removed; results are non-null; kernels use own snapshots and captured parents and terminate without exceptions. Conditional replacement accepts identity or equals equality [27, 28]; atomic compute invokes its callback once, under a bin monitor in the inspected source [26, 28]. Different keys may share bins.

Implement Table 1 using `replace(key, expected, result)` and `compute`. Validation-only callbacks return the prepared result on match, otherwise the unchanged entry with a failure flag; cached callbacks recompute on mismatch, and fresh callbacks always compute under protection. Capture permanent milestones locally rather than rereading the map.

Callbacks perform no other-key map operations, other foreground kernels or direct guard work, including on cached matches; no guard survives. Multiple preparations are allowed. Prices charge the specified calls; bin collisions add no unperformed kernel work. Assume scheduling and call completion: finite write counts do not imply fairness.

THEOREM 7 (MATCHING ACTUAL-WRITE ENVIRONMENT). *Under these conditions, with no initial preparations and at most B successful external writes,*

$$\min_P \sup_{E: \# \text{writes}(E) \leq B} \text{Cost}(P, E) = \sum_i c_i + F(J, B).$$

The equality holds for separate or shared internal bins.

PROOF. Classify job i as C exactly when some retained preparation has its current identity, otherwise U. Use Equation (4) with $b = B - \# \text{writes}$. Immediately before a comparison acquires its monitor, the environment can preserve a current preparation or invalidate it with one fresh own-key write. Theorem 6’s validation/cached inequalities apply. No monitor survives the boundary, so this works with colliding bins.

Fresh preparation performs the U-to-C credit transfer; computing from an old identity adds no current capability. Stale validation rejects without a write and leaves the semantic state unchanged.

A stale cached callback recomputes inside: its excess is $k_i + s_i - c_i = P_i$ in state U, or $k_i + s_i - m_i = w_i + P_i$ if another current preparation exists. Both dominate the protected option. Fresh callbacks use that option directly. Discarding the only current preparation increases potential; copies, stale preparations and preserving inspections cannot decrease it. Captured parents survive live-parent writes. Thus every operation admits a zero/one-write continuation preserving cost plus potential; telescoping gives the lower bound with at most B writes. The interface forbids using a shared bin to perform another job's work inside the same call.

For the upper bound, the serial controller freshly reads and prepares after every cheap failure. Its read-to-comparison intervals, including cached completions, are disjoint; each mismatch contains a distinct write. There are at most B mismatches. Counting them may overestimate remaining writes, conservatively, so the three-mode guarantee applies in either layout. $\square$

Operational accounting. Native kernels fold captured predecessor seeds and an immutable array of length $8w_i - |\text{Pred}(i)| > 0$, executing exactly $8w_i$ selected operations. The metric adds $8p_i$ per protected kernel, $8v_i$ per conditional call and $8k_i$ per callback, including mismatches. It scales the abstract objective by eight. Allocation, monitor waiting, initialization, background work and other API instructions are excluded; charges are not inferred from timing.

6 All-Budget Compilation

Direct evaluation requires up to $(B + 1)N$ states, $N \leq 2^n$. Our construction, for binary input length ℓ_{enc} , removes budget enumeration; arbitrary DAGs still retain the ideal dimension.

6.1 Reducing the Self-Referential Choice

LEMMA 8 (SUBSET MONOTONICITY AND CHEAPEST VALIDATION). *If $T \subseteq S$, then $F(T, b) \leq F(S, b)$. For any minimum- c_i job $i \in \mathcal{A}(S)$, its cheap-validation option is no larger than that of any other available job.*

PROOF. Simulate a policy for S while completing omitted jobs without invalidation. Retained-job readiness is preserved, omitted jobs spend no token and add nonnegative excess, so $F(T, b) \leq F(S, b)$. At $b = 0$ cheap values are zero. At $b > 0$, ready $j \neq i$ remains ready in $S - i$; hence

$$F(S - i, b) \leq \max\{F(S - \{i, j\}, b), c_j + F(S - i, b - 1)\} \leq \max\{F(S - j, b), c_j + F(S, b - 1)\}.$$

The branch $c_i + F(S, b - 1)$ is bounded by the same maximum since $c_i \leq c_j$, proving the comparison. $\square$

For $f(0) = 0$, set $(I_s f)(0) = 0$ and $(I_s f)(b) = \max\{f(b), s + f(b - 1)\}$ for $b \geq 1$. Fix a cheapest available i , put $g(b) = F(S - i, b)$, $c = c_i$, and

$$h(b) = \min_{j \in \mathcal{A}(S)} \{P_j + F(S - j, b), \delta_j + I_{s_j} F(S - j, \cdot)(b)\}. \quad (5)$$

At zero, $h(0) = \min_j \{P_j, \delta_j\}$ is allowed to be positive. Monotonicity and feasible completing modes give $g(b) \leq F(S, b) \leq h(b)$. Equation (3) becomes

$$f_0 = 0, \quad f_b = \min\{h(b), \max[g(b), c + f_{b-1}]\}, \quad b > 0. \quad (6)$$

This is a specialization of a reflected recurrence [30], not a new general dynamic-programming principle. Its particular child curves permit the following closed form.

LEMMA 9 (CONCAVE FLOOR AND CAP). *Suppose g, h are nonnegative, nondecreasing and discrete concave, $g(0) = 0$, and $h \geq g$. Define*

$$D(0) = 0, \quad D(b) = \sum_{t=1}^b \max\{c, \Delta g(t)\}, \quad c > 0.$$

Then Equation (6) has solution $f_b = \min\{h(b), D(b)\}$, which is nondecreasing and discrete concave.

This inherited lemma [37] follows by induction using $D(b) = \max\{g(b), c + D(b - 1)\}$. If the cap bound at $b - 1$ is strict, concavity forces its next increment below c , so the cap remains active. Flooring and pointwise minima preserve discrete concavity. The full proof is retained in the supplement.

6.2 Integer Insertion and Supporting Slopes

LEMMA 10 (INTEGER MARGINAL INSERTION). *Let a bounded curve with $f(0) = 0$ have a nonnegative supporting representation $f(b) = \min_{d \in D}(db + q_d)$ for all integer $b \geq 0$, with finite $D \subseteq \mathbb{R}_{\geq 0}$ and $0 \in D$. For $s > 0$, define*

$$Q_s = \max_{b \in \mathbb{N}} \{f(b) - sb\}.$$

Then

$$I_s f(b) = \min \left\{ sb + Q_s, \min_{d \in D} (db + q_d + (s - d)_+) \right\}. \quad (7)$$

Its marginal sequence is that of f with one additional marginal s inserted in decreasing order.

PROOF. Let t count the nonincreasing marginals of f that are at least s . Then $I_s f = f$ through t and $I_s f(b) = s + f(b - 1)$ afterwards, giving insertion and $Q_s = f(t) - st$. Each shifted line and the contact $sb + Q_s$ majorizes both portions. For attainment, an active slope at integer b satisfies $\Delta f(b + 1) \leq d_b \leq \Delta f(b)$ (only the first inequality at zero). At $b < t$ it is at least s and unchanged; at $b = t, t + 1$ the contact attains; at $b \geq t + 2$ an active line at $b - 1$ has slope below s , and its shifted line attains. This includes $t = 0$ and $f \equiv 0$. $\square$

The contact is over integers. For $f(b) = \min\{100, b + 1, 3b\}$ and $s = 2$, $I_s f(1) = 2$. Replacing the integer contact 0 by the continuous supporting envelope's supremum $1/2$ gives 2.5 in Equation (7). Computing a real contact and rounding afterwards is not the stated identity.

The floor operation also preserves a finite input-slope basis. If $f(b) = \min_d (db + q_d)$ and $c > 0$, then

$$\sum_{t=1}^b \max\{c, \Delta f(t)\} = \min \left\{ cb + Q_c, \min_{d > c} (db + q_d) \right\}. \quad (8)$$

Let $a = \#\{t \geq 1 : \Delta f(t) > c\}$, the first integer maximizing $f(b) - cb$. The floor equals f through a , and equals $cb + Q_c$ thereafter. At $b < a$ every attaining line has slope above c ; at and after a , each retained larger-slope line stays above the contact line. This definition also handles a plateau with marginal exactly c , or $a = 0$, without special rounding.

THEOREM 11 (ALL-BUDGET BASIS AND ENCODED SIZE). *For every residual set S , $F(S, \cdot)$ is nonnegative, nondecreasing and discrete concave, with eventual value $\sum_{i \in S} P_i$. It has a supporting representation*

$$F(S, b) = \min_{d \in \mathcal{D}_S} (db + q_{S,d}), \quad \mathcal{D}_S \subseteq \{0\} \cup \{c_i, s_i : i \in S\}. \quad (9)$$

There are at most $2|S| + 1$ lines and at most $4|S|$ positive marginal runs. All reachable-state curves and action data can be constructed and checked in $2^n \text{poly}(\ell_{\text{enc}})$ bit time, with no iteration through the magnitude of B .

PROOF. Induct on $|S|$, starting with the zero line for the empty set. By Lemma 10, each cached term in h shifts existing intercepts and adds only slope s_j . Protected terms shift intercepts by P_j . Their minimum is a concave supporting envelope. Lemma 8 gives $g \leq F \leq h$ independently of the desired formula, so Lemma 9 applies. Equation (8) adds only the cheapest available c_i . Union the

line families and retain the smallest intercept per slope. This proves Equation (9), concavity and nondecreasingness.

Protecting all jobs gives the bound $\sum_i P_i$. A bounded finite lower envelope of nonnegative slopes is eventually constant; let its limit be L_S . By smaller-set induction, sufficiently large budgets in Equation (3) satisfy

$$L_S = \min_{i \in \mathcal{A}(S)} \left\{ P_i + \sum_{j \in S-i} P_j, \quad c_i + L_S, \quad \delta_i + s_i + \sum_{j \in S-i} P_j \right\}.$$

Here $c_i > 0$ and $\delta_i + s_i = P_i + w_i > P_i$, forcing $L_S = \sum_{i \in S} P_i$.

At most $2|S|$ distinct positive slopes give at most that many continuous-envelope changes. Integer sampling adds at most one bridging unit interval per noninteger change, so at most $4|S|$ positive runs suffice. This is a conservative upper bound, not a tightness claim. Summing $4|S| + 1$ over all subsets gives $(2n + 1)2^n$ profile entries, with at most $n2^n$ action identities.

Intercepts obey $0 \leq q_{S,d} \leq \sum_{i \in S} (P_i + w_i)$: protected/cached shifts add at most $P_i/P_i + w_i$, and contacts are bounded by child saturation. For affine profile triples (a, y, d) , contacts satisfy $Q_S = \max(y - sa)$ over listed breakpoints; the constant tail decreases after subtracting sb . Thus intercepts, crossings and integer neighbors have polynomial bit length in ℓ_{enc} . Each state uses polynomially many operations on $O(n)$ lines; the checker partitions at the same crossings. At most 2^n states give the bound.

□

6.3 Emitted Policies and Independent Checking

Profiles store affine pieces (a, y, d) and a constant tail. Constructors process reachable upsets child-first. Runtime evaluates Equation (3) with indexed profiles, selects an argmin and decrements allowance on mismatch; queries are polynomial in n , stored size and $\log(B + 1)$.

A checker importing no constructor reconstructs readiness, reachable sets, raw/derived prices, baseline and action identities. It checks origins, continuity, decreasing slopes, tails and Bellman equality. Partition at own/child starts and their one-step shifts; all branch expressions are affine within a part. Endpoints and floor/ceiling neighbors of crossings certify each finite part; check the constant tail directly. The same method compares supporting bases with emitted curves. Induction on $|S| + b$ then certifies values and the mathematical argmin. Executable selection, arithmetic, parsing and abstraction remain trusted or separately tested. Arbitrary certificate encodings additionally cost their supplied length.

What a requested-value query can skip. Large b need not require a profile. Classical concave allocation [14] gives the following charged-model bounds:

$$\max_{\substack{b_i \in \mathbb{N} \\ \sum_i b_i \leq b}} \sum_{i \in S} \min(P_i, b_i c_i) \leq F(S, b) \leq \min_{t \in \{0\} \cup \{c_i : i \in S\}} \left(\sum_{c_i > t} P_i + bt \right). \quad (10)$$

For the lower bound, assign private allowances b_i : protected/cached-failure completion pays at least P_i , while other completion after exhausting b_i failures pays $b_i c_i$. For the upper bound, protect $c_i > t$ jobs and retry the rest. Allocation merges $\lfloor P_i/c_i \rfloor$ copies of c_i and a remainder without enumerating b ; equality certifies a value, and $b \geq \sum_i \lceil P_i/c_i \rceil$ certifies saturation. Requested-value methods use these tests per subproblem; their service differs from all-budget profiles/routing.

7 Compact Compilation and Its Limits

7.1 Reducing Mixed Charges to Two Fees

For a concave suffix f with $f(0) = 0$, write

$$R_{a,q}f(b) = f(b) + \min \left\{ q, \sum_{\ell=1}^b (a - \Delta f(\ell))_+ \right\}. \quad (11)$$

This is the two-fee operator with failure price $a > 0$ and protected premium $q \geq 0$: by Lemma 9, it equals $\min(f + q, L_a f)$, where $L_a f(b) = \max_{0 \leq j \leq b} \{f(j) + a(b - j)\}$. It is monotone in f .

LEMMA 12 (TWO-FEE ORDERING AND PACKING). *Independent two-fee jobs admit a fixed optimal nondecreasing- a completion order, including against adaptive choices. Composing their R operators in this order gives their value H . Reverse composition fills marginal deficits below the next a : all positive suffix marginals except possibly the last are at least a . Each job adds at most two runs. Every sorted suffix is the root curve capped at its remaining premium sum.*

The inherited proof [37] applies Lemma 8's minimum-failure-price comparison to the two-fee recurrence and inducts on $|S| + b$. Reverse composition raises a final partial marginal, then appends full a -columns and at most one remainder. Removing the new tail mass gives the suffix cap. The supplement retains the complete ordering and packing proof; the charged reduction below is additional.

THEOREM 13 (EXACT MIXED-PRICE REDUCTION). *Suppose $p_i \delta_i = 0$ for every job, with the original prices from Equation (2), and set*

$$t_i = \begin{cases} \min(c_i, s_i), & \delta_i = 0, \\ c_i, & \delta_i > 0 \quad (\text{hence } p_i = 0). \end{cases} \quad (12)$$

For independent jobs, the charged excess $F(S, b)$ equals the two-fee optimum $H(S, b)$ with prices (t_i, P_i) at every integer budget. A fixed nondecreasing- t completion order is optimal. The same equality holds for a DAG satisfying $t_u \leq t_v$ on every edge $u \rightarrow v$, with ties ordered topologically. The total charge remains $\sum_{i \in S} c_i + H(S, b)$.

PROOF. Marginal insertion (Lemma 10) commutes with deficit filling when $s \geq a$:

$$R_{a,q} I_s f = I_s R_{a,q} f. \quad (13)$$

Indeed, insertion of s precedes every marginal below a and adds no deficit; $R_{a,q}$ changes only those later marginals, raising them at most to a .

If f is a sorted suffix with effective costs at least t , $s \geq t$ and $q < s$, then $R_{t,q} f \leq I_s f$. Before the final partial marginal the deficit is zero. At that marginal the increment is at most $t - \Delta f(b) \leq s - \Delta f(b)$; after saturation it is at most $q < s$. These cases also show $R_{s,q} f = \min(f + q, I_s f)$ when $t = s$.

For $\delta_i > 0$, $p_i = 0$, so cached completion is dominated: $\delta_i + I_{s_i} f \geq P_i + f$. The ordered charged operator is exactly R_{c_i, P_i} . For $\delta_i = 0$, $P_i = p_i < s_i$, and it is $\min(f + P_i, L_{c_i} f, I_{s_i} f)$. The preceding suffix inequalities make this R_{t_i, P_i} , both when $t_i = c_i$ and when $t_i = s_i$. Thus the sorted charged policy attains H .

For a matching lower bound, the two-fee Bellman equation bounds $H(S, b)$ by $P_i + H(S - i, b)$ and by $\max\{H(S - i, b), c_i + H(S, b - 1)\}$, since $t_i \leq c_i$. These cover protected and cheap choices. Positive- δ_i cached choices are covered by protection dominance. For a zero- δ_i job at any sorted position, write $H(S) = A R_{t_i, P_i} f$ and $H(S - i) = A f$. Every operator in the preceding composition A has cost at most $t_i \leq s_i$. Monotonicity, the suffix inequality and Equation (13) give

$$H(S) \leq A I_{s_i} f = I_{s_i} A f = I_{s_i} H(S - i).$$

Hence every original charged Bellman option is at least H ; induction on $|S| + b$ proves the lower bound against adaptive choices. Finally, removing DAG edges gives an independent lower bound, and a compatible sorted topological policy attains it legally. $\square$

The class mixes $k_i \leq v_i$ jobs with zero-premium jobs. With $v_i = k_i = \kappa$ and $p_i = \lambda w_i$, $t_i = \min(w_i + \kappa, (1 + \lambda)w_i)$ increases with w_i , so work order suffices. Effective prices preserve cached choices; the total-cost baseline remains $\sum c_i$.

7.2 A Checked Cursor Policy

Minimum- t Kahn traversal finds a sorted topological order when the edge condition holds. Apply Lemma 12's packing to (t_i, P_i) , retaining the original charged input. For sorted suffix S_k , root curve H and $Z_k = \sum_{i \in S_k} P_i$, store

$$F(S_k, b) = \min\{H(b), Z_k\}. \quad (14)$$

The artifact stores $O(n)$ entries beyond the graph for its $n + 1$ policy suffixes; the general artifact covers all reachable unfinished sets.

The checker recomputes reduction, baseline and order, then checks decreasing integer runs and mass $H(\infty) = Z_0$. For adjacent caps $a = Z_{k+1} \leq z = Z_k$, partition at their first reaching budgets: verify the uncapped prefix, each boundary, constant marginal t_i between them and saturation. Concavity extends endpoint checks; zero premiums require identical adjacent caps. Suffix induction verifies the two-fee equations; Theorem 13 supplies charged/global optimality. Original-mode boundary controls do not substitute for this proof.

Construction takes $O(|E| + n \log(n + 1))$ arithmetic operations. Checking r supplied runs takes $O(|E| + n \log(r + 2) + r)$ plus encoding length; generated roots have $r \leq 2n$. Binary operand lengths count. Indexed queries cost $O(\log(n + 1))$. The cursor evaluates all three original branches for its fixed next job: cheap failure stays, cached failure spends a token and advances. All charges remain original; inputs outside the sufficient class use the general compiler.

The conditions are sufficient. For independent $w = (1, 1, 1, 2)$, $p_i = w_i$, common $v = 5$, $k = 6$, and $B = 1$, ascending work order costs 31, while order $(2, 1, 1, 1)$ and the adaptive optimum cost 30; here $p_i \delta_i > 0$. Even with $\delta_i = 0$, sorting by c_i can fail: $(w, p, k, v) = (5, 6, 5, 8)$, $(5, 4, 6, 8)$ gives $c = (10, 11)$ but $t = (10, 9)$, and $B = 1$ totals 31 versus 30. Both refutations remain in the artifact.

7.3 A Sharp Bound on Independent-Job Savings

The third mode's scalar benefit is bounded even outside the compact class. Write F_2, F_3 for the two- and three-mode excess values and $A = \sum_i c_i$.

THEOREM 14 (SHARP INDEPENDENT-JOB RATIO). *For independent jobs with arbitrary charged prices and any integer $B \geq 0$,*

$$\frac{A + F_2(B)}{A + F_3(B)} \leq \frac{1 + \sqrt{2}}{2}.$$

The constant is a supremum, already approached with common $v_i = k_i$ and $p_i = \lambda w_i$.

PROOF. Put $X = F_2(B)$ and $Y = F_3(B)$. Equation (10)'s private-budget argument gives $D(B) = \max_{\sum b_i \leq B} \sum_i \min(P_i, b_i c_i) \leq Y$, including when $\delta_i > 0$.

Pack the two-mode premiums in reverse sorted- c order (Lemma 12), labelling each contribution by its job. A final unit column containing job i 's mass has height $h \leq c_i$, since later caps are no larger. If x_i is that job's mass in the first B columns, then $\sum x_i = X$, $0 \leq x_i \leq P_i$, and $\sum x_i / c_i \leq B$: each nonzero column contributes at most its mass divided by its height, namely one.

Let ϕ_i linearly interpolate $\min(P_i, bc_i)$ at integer b . Under $z_i \geq 0$ and $\sum z_i \leq B$, the maximum of $\sum \phi_i(z_i)$ is $D(B)$: select the B largest nonincreasing unit slopes, preserving tied prefixes. For

$z_i = x_i/c_i = \ell_i + \rho_i$, $\ell_i \in \mathbb{N}$, $0 \leq \rho_i < 1$, $z_i \leq P_i/c_i$ implies $\phi_i(z_i) \geq c_i(\ell_i + \rho_i^2)$. With $\alpha = \sqrt{2} - 1$,

$$\ell_i + \rho_i^2 - 2\alpha(\ell_i + \rho_i) + \alpha^2 = \ell_i(1 - 2\alpha) + (\rho_i - \alpha)^2 \geq 0.$$

Thus $Y \geq 2\alpha X - \alpha^2 A$; $1 - \alpha^2 = 2\alpha$ gives the ratio. For sharpness take $n = M^2$ equal jobs with $w = 1$, $v = k = M$, $p = \lfloor \alpha M \rfloor$ and $B = \lceil np/(M+1) \rceil$. Packing gives $F_2 = \min(np, B(M+1))$ and $F_3 = \min(np, B(p+1))$; their total-cost ratio converges to the bound. $\square$

Hence the supremum relative total-cost saving is $3 - 2\sqrt{2} \approx 17.16\%$ on independent inputs. The argument uses sorted independent-job packing; it asserts no such bound for arbitrary DAGs.

7.4 Hardness Outside the Compatible Class

With $v_i = k_i = 0$, cached completion is dominated by a cheap attempt followed by immediate protection. The inherited zero-fee result [37] makes deciding $F(J, 1) \leq K$ NP-complete even for two DAG layers, indegree at most two, unit premiums and two positive failure prices. Its proof reduces CLIQUE [36] by choosing which vertex jobs to protect before endpoint-dependent jobs; the full construction and fixed-order equation remain in the supplement. This establishes neither NP membership for binary B nor necessity of ideal enumeration. A w -chain cover gives at most $\prod_{\ell} (n_{\ell} + 1)$ ideals, polynomial for fixed w .

Dependencies can require adaptive ordering without fees: $c = (3, 7, 5, 1)$, $p = (2, 2, 6, 2)$ and edges $0 \rightarrow 1, 0 \rightarrow 2, 1 \rightarrow 3$ give 24 versus 26 at $B = 2$. Every fixed order gets adaptive modes and knows B . This constructed separation does not establish typical benefit.

8 Source Integrations and Their Boundaries

Roslyn document updates without a writer budget. The `Workspace.cs` loop at commit 6c4a46a3 transforms a captured immutable solution outside a semaphore, validates identity under it and retries mismatches. Publication preserves notification ordering [2, 3]. We modify only the `SourceText` path. After r global failures, two-mode completion transforms the latest solution inside; three-mode completion prepares outside, then reuses on match or recomputes inside on mismatch. The selector receives no writer/price tables.

Replayability and safety under protection are separate obligations. The scratch document list must be cleared each attempt; source review discusses replay and event ordering [3]. Our initial two-mode patch deadlocked on same-text notification reentry; restoring the pre-lock identity no-op repairs it. Four later missing-document cases timed out when virtual `GetDocumentName` reentered an unrelated update while holding the semaphore. Patch04 detects an absent target and runs the original transformation on the same captured solution after releasing protection, preserving errors/exception handling. All 14 fixed error cases then complete. These defects belong to our patches.

RQ2 checks linked propagation, event payloads and snapshots. Variable/lazy work, allocation and `VersionStamp` generation limit transfer to counts. Custom loaders, changed-text notification reentry and arbitrary compiler behavior remain outside scope. No upstream numerical call cap or latency SLA was found; r is authored. A stronger prepared-rebase comparator reuses existing `WithDocumentContentsFrom` when solution ID, related-document IDs and all affected project states remain unchanged. It merges prepared contents into the latest solution under protection, otherwise using three-mode fallback. This partial operation lies outside our whole-kernel interface.

Deephaven with owned refreshes. For `ConstructSnapshot/hierarchical-table` sources at 6367313a, the server API materializes key directives K followed by viewport V [6]. Fixed keys/hierarchy, stated Action changes and a 16-row/four-column viewport yield source-derived selected-operation bounds $W_K = 4$, $W_V = 104$, including mid-body updates; the full derivation is supplementary.

Completed K persists for V after live Actions change. With protected weight $1 + \lambda$, supply $p_i = \lambda W_i$ and zero call charges. This excludes allocation, waiting and unselected API instructions.

The caller owns at most q public refresh requests [8]; synchronized START/completion boundaries charge at most $2q$ failed bodies. Periodic updates do not supply such a bound [7], and partially notified interleavings are excluded. Imported compilation gives an upper bound: at $\lambda = 3$, $q = 2$ a script attains 424 versus a same-budget local rule’s 432, but a phase-aware game gives 420. The abstract game is therefore not the native system’s exact minimax game.

9 Implementation and Evaluation

We test game/policy agreement and collateral blocking (RQ1), source integrations (RQ2), and construction costs (RQ3). Studies are authored and exploratory: reused cases are identified and new generation rules precede outcomes. Every input, failure, timeout, invalid unit and correction remains recorded; none is a held-out application population.

An Apple M2/24 GiB runs Python 3.10.9 (later 3.12.14) and OpenJDK 17.0.19; the inspected map source matches the cited version, but its binary was not rebuilt. Hashes bind inputs, code, commands and traces. Codex assisted design, inputs/checkers, source comparison, proofs, analysis and writing; Claude informed study direction. Traces come from archived executions, not AI-generated measurements. These author-side checks are neither mechanical proofs nor independent blind review; no human or simulated-human study was performed.

9.1 RQ1: Charged Values, Routing and Native Calls

Table 2 separates proof-oriented checks from native observations. Inputs are authored and some were observed during development; they are not independent application samples. General and compact values are serialized, reloaded and compared with direct recurrences. Separate checkers reconstruct routing, prices, counters and policies from saved records. Numeric-alias, null-input, epoch, output and writer-order defects, their original failures and correction checks remain in the supplement. Native arithmetic is bounded.

Table 2. Distinct finite checks, all agreeing after recorded repairs. Rows overlap; full parameters, old failures and controls remain in the artifact.

Obligation	Inputs and checked outcomes
Priced value/routing	3,183 general inputs; 19,485 compact/fallback inputs (15,727 observed, 3,758 new), 527,409 branches.
Native scalar accounting	34 inputs, 2,278 paths, 816 concurrent runs, 340 maxima; compact: 610 records, 208 maxima, 30/36 screened inputs inapplicable.
Resource frontier	2,442 native records/576 maxima; direct equations on 265,629 budget/slack pairs of 5,421 DAGs; aware/unaware native records 9,314/17,714 and maxima 1,024/1,280.
Threshold work choice	86,662 fixed orders/2,773,184 pairs on 5,519 inputs; adaptive-family equations on 176,608 roots.
Suffix existence	Separate finite-rank AND-OR recursion: 665,500 states, 363 weight tuples; suffix Java: 20,594 paths/384 maxima.
Independent-job ratio	13,295 inputs/172,835 roots and seven tightness families; another 622 inputs at three scales, 24,378 roots and 3,732 large-budget queries.

The native unaware selector receives no writer/price/table state; tested budgets reach four. Separate counters match W , L , Q and $R = W + L + 8\kappa Q$. Scalar three-mode gains occur in only 2/96

cells (72 versus 80); 94 tie. A decreasing five-job chain has scalar-optimal $L = 48$ versus cached-all's 40 at equal $Q = 5$ and worst cost 200: scalar optimality need not minimize protection. Repaired precondition checks cover all 96 tables/2,442 saved decisions and five unrelated-error controls.

All-tail checks cover 5,520 inputs/176,640 roots: 285 strict worst- W improvements and 176,355 ties, all preserving the frontier; 2,552 Java paths match 144 maxima. Adding 1,024 five-job chains yields 6,544 inputs/209,408 roots: greedy certified choice is better/worse/tied versus all-tail at 1/3/209,404. Both native checkers reject eight corruptions. Later constructed counterexamples also refute global W -optimality of these two rules and simultaneous continuation optimality; they do not refute suffix *existence* or threshold-family optimality. The ratio family at 65,536 jobs reaches 1.20545; the proof supplies the limit. Scaling is not independent sampling.

Calibration and controlled blocking. JMH 1.37 [4] tests nine wrappers, four payloads and two layouts: all 72 cells/144 forks/720 measurements complete, with three 200-ms warmups and five 200-ms measurements per fork. A nonnegative w, v, k, p fit admits only 2/8 settings at maximum residual $\leq 20\%$; all fitted p are zero. None of 72 admitted scalar roots gains, and 1,080/1,152 roots remain outside. Short warmups, reuse and lifetime differences prevent a mean-time claim.

A separate Java probe uses four dependent kernels $(1, 2, 3, 5)s$, $r = 0, 1, 2$ and exactly $B = r$ initial failures. Both unaware policies use $Q = 4+r$, $W = (11+r)s$; two/three modes protect 11s/0 iterations. A budget-aware two-mode policy could also avoid protection. At each completing callback, an unchanged-value compute on another key signals readiness before entering; the foreground holds its bin until that signal. Same/disjoint-bin layouts share the hook. Four scales, three allowances, two layouts/modes and three JVMs with 30 repetitions give 4,320 measurements/1,440 warmups, all complete, matching 12 output references. A repaired cross-job checker validates all saved records and rejects 12 corruptions. Summed waits at the largest scale fall from 26.8 ms to 31.2 μs in shared bins; disjoint differences are submicrosecond and sometimes reverse. All 24 cells remain. This is forced collateral blocking, not natural-arrival performance, WCET or blocking of get.

9.2 RQ2: Roslyn Semantics and Arriving Updates

Source projection and semantic checks. Pinned net8 Roslyn assemblies use a pinned .NET 10 RC runtime. Four projects' evaluated Compile items yield 3,837 files, 4,019 document occurrences and 40.8 MB, excluding two package-external items before outcomes. Fixed targets maximize sharing then size (63,593 bytes, two projects) and single-project size (background: 977,535 bytes). This source projection has internal references; updates append comments.

The 174-case grid covers $r = 0, 1, 2$, concentrated/distributed writers, same/linked targets, no-ops, distinct equal-content texts and missing/deleted targets. Patch04 plus four unmodified controls pass: 1,645 events, 941 publications, 1,975 snapshots and ten rejected corruptions. A separate checker binds immediate/queued payloads and callbacks to their solutions, checks publication chains and linked transitions, and preserves 4,016 untouched text identities/versions and checked metadata.

Adding 58 rebase cases yields 236 passing cases, 2,190 events, 1,253 publications, 2,653 snapshots and 12 rejected corruptions. Its $B = 5, 6$ cases concentrate initial writes to realize B within fewer calls: placement changed before outcomes; interleavings differ. Error-reentry/same-text controls pass 18/5 cases versus patch04's 14/4. Earlier 204 patched/16 unmodified scale checks, initial 3,960 measurements, same-text deadlock and four missing-document timeouts remain. These projections do not establish full concurrent equivalence. A separate 96-process, four-project study changes types/constants, unrelated source, parse options, references or linked text across eight arms and cold/warm caches. All 2,556 project projections match declared sources, types/constants, diagnostics and emission; eight corruptions reject. Workspace/direct compilations share Roslyn; independent expectations cover only these authored programs.

Retained forced-interference measurements. Patch03 uses four updates/batch, $r = 0, 1, 2$, deduplicated $B \in \{0, r, r+1, r+4\}$ and three processes with two warmup/ten measurements. Writers use the first B pre-lock opportunities. All 990 measured/198 warmup units pass the old checker (11 corruptions), lacking payload validation. Across 330 measured batches/mode, original/two/three calls are 2,040/1,590/1,590, protected transformations 0/1,080/450 and total transformations 2,040/1,590/2,040. Three-mode foreground medians beat each comparator in four settings and lose in seven. New checks do not replace these timings.

Independent arrivals and a stronger source comparator. Each batch serves 32 serial foreground replacements while a producer independently enqueues 64 background replacements at jd from a shared epoch, $d \in \{0, 10, 100, 1000\} \mu s$. A single FIFO worker invokes the original background API. Both threads signal readiness before the epoch; texts are precomputed and construction costs are separate. The pre-lock collision hook is null. The counter resets per batch without arrival/write inputs.

An initial six-arm study (unmodified, instrumented original, two/three modes at $r = 0, 2$) completes 864 units (720 measured/144 warmup), but its shuffle places all original processes first. A new six-block cyclic Latin design balances position: all 1,728 units (1,440/288) pass. At $100 \mu s/r = 2$, three/two background p95 medians are $73.33/265.35 \mu s$, lower in all six blocks; foreground directions are mixed. At $r = 0$ background improves in only five blocks, reversing by 1.889 times in the sixth. Burst foreground is slower with three modes in all six blocks. Instrumented-original foreground overhead is 6.9–13.8% (12.7–24.4% initially). All outcomes remain.

Adding rebase at $r = 0, 2$ retains every interval. Eight cyclic Latin blocks give 64 processes, each with two warmup/ten measured batches per interval: all 3,072 units (2,560/512) pass, without failures/timeouts/invalids/exclusions. Instrumented arms share one binary. The checker validates timelines, linked payloads, 97 snapshots/batch and resource identities; ten corruptions reject. Figure 2 shows all 32 cells.

All capped batches meet $Q \leq 32 + r$. Across 320 measured batches/arm, $r = 0$ two/three modes both use 10,240 calls; full protected transformations fall from 10,240 to 4,274, while full transformations rise to 14,514. At $r = 2$, calls total 10,802/10,786, full protected transformations 7,131/3,641 and full transformations 10,802/14,427. Different interleavings need not yield equal aggregate calls. Rebase uses 10,240 calls and outside transformations at either allowance, with zero full protected transformations but 4,219/4,528 protected merges. Here every background update is in an unrelated project, so every rebase mismatch is eligible. Zero full protected transformations does not mean zero protected work.

At $100 \mu s/r = 2$, three/two background p95 medians are $60.00/306.10 \mu s$, lower in seven/eight blocks; foreground medians are $693.98/666.38 \mu s$, with mixed block directions. At $0/10 \mu s$, their foreground medians are $993.54/603.88$ and $888.10/653.00 \mu s$. Rebase/three foreground medians at $100 \mu s$ are $680.96/713.35$ for $r = 0$, but $792.83/693.98$ for $r = 2$; background medians similarly reverse, $63.08/85.17$ versus $88.02/60.00$. The eligible rebase arms have the same behavior yet different timings. Process variability and merge cost preclude a stable speedup inference from full-transformation counts. This exposes stronger operations outside the abstract optimum.

Deephaven boundaries. All 216 prefix/END cells succeed, with 153 cycles/223 inconsistent bodies but no prefix abort: ExpandAll bypasses the check. Compiler/local wins occur in only 3/36 cells; six lose to the upstream replica and four to the unknown-budget threshold. Twelve public-graph cells expose policy-dependent refresh counts. Changing Actions gives 28 full/eight contracted results; two retain old directives after re-expansion. The repaired checker rejects 17 corruptions (eight missed earlier); ten regressions and 108 phase-aware cells pass. Full charges, source conditions and corrections remain; native minimax is unestablished.

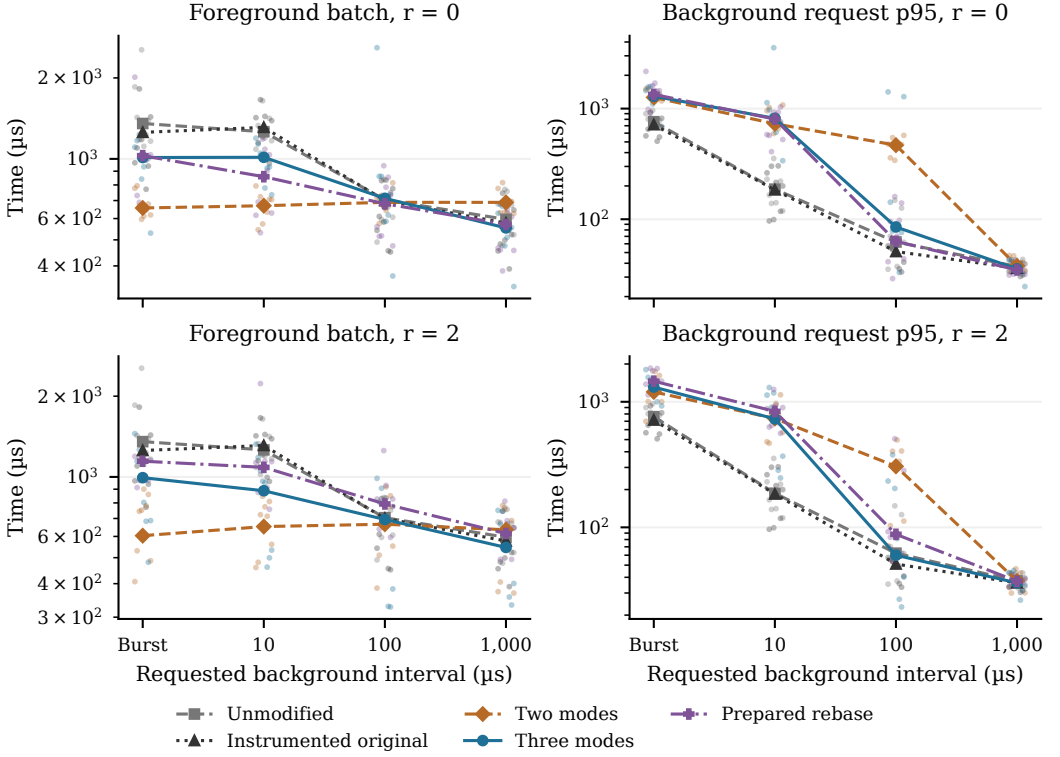


Fig. 2. Roslyn independent arrivals including prepared rebase. Lines are medians of 80 batches per cell; faint dots are eight process medians. Background delay is each batch’s nearest-rank p95 over 64 requests from intended arrival to return. All four intervals and both retry allowances are shown. These authored arrivals do not establish a production latency bound.

9.3 RQ3: Construction Cost and Stronger Value Queries

For 24 DAGs ($n = 8, 12$), three price scales and 1/4/12 queries through 2^{64} , ALL builds/reloads/checks profiles, PEAK certifies requested values and DP uses two Bellman rows. Imports-through-checking caps are 4 seconds/1 GiB. Initial ALL/PEAK/DP successes are 63/62/64 of 72, with eight AND-OR successes. Stronger bounds raise PEAK to 64; others time out. Shared values agree. On 14 common 12-query successes, PEAK/ALL median time falls from 7.16 to 0.576; DP/ALL is 0.457. Only ALL finishes seven long-premium cases; its nine timeouts have 12 independent jobs.

Compatible-class scaling. Campaign A fixes 42 inputs ($n = 8..32768$), independent/height-two DAGs, three price scales and 1/12 queries, with 8-second/1-GiB caps. All 420 outcomes are retained (Table 3); PEAK/DP also validate adjacency. PACK’s 12 largest-input runs take 0.837–1.151 seconds, peak RSS 122,044,416 bytes; median PEAK/PACK and DP/PACK ratios are 0.860/0.792.

A has only 40 runs in its large curves. Campaign B uses 18 inputs at $n = 512, 4096, 32768$, exactly $2n$ runs, long plateaus and queries through 2^{64} . All 180 outcomes use the same caps. At 32,768 jobs, PACK constructs/checks 65,536 runs in 1.333–1.603 seconds, peak RSS 152,911,872 bytes; median PEAK/PACK and DP/PACK ratios are 0.301/0.206. Shared values agree. An aggregation repair uses saved raw records without rerunning. These structural samples do not establish prevalence; requested values can be cheaper.

Table 3. Compact-class campaigns. Each cell is success/timeout/memory limit. A: 84 units per method; B: 36 units per method with $2n$ root runs. All outcomes are retained; returned services differ.

Method	A	B	Returned service
PACK	84/0/0	36/0/0	Checked all-budget cursor policy
ROOT	84/0/0	36/0/0	Same-reduction values, no checker
ALL	22/56/6	0/30/6	Checked all reachable unfinished sets
PEAK	34/50/0	18/6/12	Checked requested root values
DP	34/46/4	18/15/3	Requested values only

Other retained comparisons give 7,866 successful charged-order roots/874 inputs: only the known 24-versus-26 example gains from adaptation, none among 840 new independent inputs. Sixteen post hoc cases refute c order. Zero-fee gains occur at 2/192 roots; CP-SAT/screened DP/full construction succeed on 78/68/53 roots, with remaining roots timing out.

10 Related Work

Synchronization synthesis and fallback. Kung and Robinson [1], Section 3.3/p.220, already retain the critical-section semaphore during reexecution after repeated validation failure. The mechanism is inherited; Theorems 1–2 characterize its resource boundary here. Černý et al. [20] synthesize retries/guards/caches from partial programs with a scheduler and weighted limit-average safety objective. Their Theorem 6 matches action distributions and weights in quantitative probabilistic bisimulation; the erratum corrects complexity to PSPACE. Our potential is an initial worst-total-cost lower bound, not intermediate-state equality.

Their later work [21], Sections 6.1–6.2, already optimizes protected-statement counts and statistically profiles acquisition/contention under an oblivious scheduler. Here a universal call cap is coupled to worst protected work for every actual-write bound, hidden from the selector. A fixed finite instance can be expressed as a general game; the all- B policy and atomic-call lower bound require additional arguments. Preemption-safety synthesis [22] guarantees deadlock-free lock placement; our source integration has bounded checks. Worst fault-cost reachability [9] and resilience/cost synthesis [11, 12] are established.

Reduction and robust dynamic programming. Gangamreddypalli et al. [23] preserve failures and reach original successful final states or failure. Our calls do not commute unconditionally with same-key writes; the potential gives no exact intermediate-state quotient. A cost-preserving reduction remains unestablished. Mannor et al. [10], Section 4/Theorem 5, provide finite global-deviation-budget DP: unfinished upsets, completing nominal branches and token-consuming failures instantiate it with negated cost and horizon $n + B$. Postek et al. [33] supply exact finite-candidate DP, state expansion and causal budgeted routing. Normal form and all-budget closure need additional proof.

The zero-fee package [37] is an earlier snapshot of this same research, not independent prior work. We reuse its hardness, floor/cap lemma and two-fee ordering/packing. Its lineage map distinguishes the present atomic-call potential, universal frontier, second-slope closure, sharp ratio and mixed-price reduction.

Envelopes, reflection and allocation. Burdzy et al. [30], Equations (2.12)–(2.13)/Theorem 2.6, give the general reflected recurrence; our floor/cap lemma specializes it to concavity. Marginal insertion is elementary. Bertsimas and Sim [34], Theorem 3/Equations (15)–(19), already give an input-deviation slope envelope $Z(\Gamma) = \min_t \{\Gamma d_t + F_t\}$. Such envelopes alone are not new. Here Equations (7)–(9) close the operations for repeated causal failures, two completing modes and dependent job

choices. Reduction to a fixed binary robust set remains unestablished. Concave allocation [14, 15] supplies lower bounds; decision-dependent information and controllable uncertainty are broader settings [17, 18].

Scheduling, hardening and ordering. With fixed one-failure modes, zero-fee jobs map to Lawler’s precedence sequencing [31]; hardness concerns choosing modes. Fault-tolerant DAG schedules [24] and stochastic fixed-priority hardening [13] impose timing/signalling constraints absent here. Chrobak et al. [16], Lemma 4/Theorem 5, compute prescribed-order latest completion in $O(n)$ independently of the fault bound, but mandate retry. Bold and Goerigk [35] use $\Gamma + 1$ layers after duration observation; their activities complete despite delay, whereas our failure leaves a job unfinished before another choice.

Ski-rental thresholds/orderings [25, 29] require a resource-preserving map. The direct substitution $f(H) = \sum_{i \in H} p_i$, $g(H) = \sum_{i \in H} c_i$, $T = B$ gives 6 for $(p, c) = (2, 5), (4, 6)$ at $B = 1$, versus our value 5; only that map is excluded. At $B = 1$, charged fixed-order values compose $\psi_i(x) = \min\{x + P_i, \max(x, c_i), \delta_i + \max(x, s_i)\}$. Kawase et al. [5] provide the general composition-ordering connection, but not the present all-budget closure.

11 Limitations and Threats to Validity

Partial repair changes the information boundary. With two unit components, two calls and single-component writes, each call repairs dirty components and completes, or rejects and reprepares outside. Known- B minimum worst repair costs at $B = 0, 1, 2, 3$ are 0, 0, 1, 1: first accept at most $\lfloor B/2 \rfloor$ dirty components; otherwise repair last. A B -unaware policy optimal at $B = 1$ must reject a one-component first mismatch. Two further writes, one per component, force cost 2 at $B = 3$, precluding simultaneous optimality. The artifact retains a footprint-coverage proof and 3,495 inputs/160,080 checks; these are not timing bounds.

Other exclusions are replenished budgets, structural changes, exceptions and external completion. Scalar minimax needs supplied B and prices; knowing writers have finished reduces cost to baseline.

Exposed guards change the interface. The prior zero-fee model allows retained/batched guards and two-mode equality. With charged failed acquisitions, five of 384 inputs improve strictly (including 242 versus serial 243); immediate release/completion also fails at an intermediate state (43 versus 45). Charged exposed-guard equality from all-unprepared states remains unproved; our theorem does not resolve it.

Roslyn’s two patch deadlocks distinguish replayability from safe protected execution. Finite projections establish neither arbitrary-host safety nor editor throughput; rebase retains protected merge cost. Deephaven needs owned refreshes and synchronized boundaries; its abstraction is not exact native minimax. Whole-system optimality and prevalence remain unknown.

Tests do not mechanically verify proofs. Scalar gains are sparse, calibration fails and direct methods can cost less. Java arrivals are forced; Roslyn arrivals are independently enqueued but authored on one host, with instrumentation overhead and process variability. Human benefit was not measured.

12 Conclusion

One policy meets a call cap and minimizes worst protected work for every hidden writer budget within the whole-kernel interface. Suffix certificates and priced compilation support further choices. Roslyn demonstrates conditional background-delay gains, foreground burst costs and the need for safe protected execution.

Data Availability

The charged supplement [38] contains complete fixed inputs, proofs, sources, raw outcomes, corrections and replay commands; prior zero-fee evidence [37] is unchanged. Original/public hashes record path projections. Licensed modified sources retain notices; upstream archives, runtimes, caches, articles and private correspondence are excluded.

References

- [1] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [2] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. *Workspace.cs*, *SolutionCompilationState.cs* and *SolutionState.cs*. Accessed September 9, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [3] .NET Foundation contributors. 2020. Add *SetCurrentSolution* with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [4] OpenJDK. n.d. *JMH: Java Microbenchmark Harness*, version 1.37. Accessed September 9, 2026. <https://github.com/openjdk/jmh/tree/1.37>.
- [5] Yasushi Kawase, Kazuhisa Makino, and Kento Seimi. 2016. Optimal Composition Ordering Problems for Piecewise Linear Functions. In *ISAAC, LIPIcs* 64, 42:1–42:13. <https://doi.org/10.4230/LIPIcs.ISAAC.2016.42>. Extended version: *Algorithmica* (2018), <https://doi.org/10.1007/s00453-017-0397-y>.
- [6] Deephaven Data Labs. 2026. *Deephaven Core source*, commit 6367313a319b79437d4d2116c77836ad56d5138d. *Files ConstructSnapshot.java*, *HierarchicalTableImpl.java*, *EventDrivenUpdateGraph.java*, and *JsTreeTable.java*. Accessed September 9, 2026. <https://github.com/deephaven/deephaven-core/tree/6367313a319b79437d4d2116c77836ad56d5138d>.
- [7] Deephaven Data Labs. 2026. *tree: Real-time tree table*. Python documentation, version 0.40.7. Accessed September 9, 2026. <https://deephaven.io/core/0.40.7/docs/reference/table-operations/create/tree/>.
- [8] Deephaven Data Labs. 2026. *Execution context*. Official documentation, custom update graphs. Accessed September 9, 2026. <https://deephaven.io/core/docs/conceptual/execution-context/>.
- [9] Emil Dumitrescu, Alain Girault, Hervé Marchand, and Éric Rutten. 2007. *Optimal discrete controller synthesis for the modeling of fault-tolerant distributed systems*. INRIA Research Report RR-6137. https://www.researchgate.net/publication/250079523_Optimal_discrete_controller_synthesis_for_modeling_fault-tolerant_distributed_systems.
- [10] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [11] Daniel Neider, Alexander Weinert, and Martin Zimmermann. 2020. Synthesizing optimally resilient controllers. *Acta Informatica* 57, 195–221. <https://doi.org/10.1007/s00236-019-00345-7>.
- [12] Alexander Weinert. 2019. *Optimality and Resilience in Parity Games*. PhD thesis, Saarland University. Section 6.2. https://finkbeiner.groups.cispa.de/_assets/paper.C89PMq4G.pdf.
- [13] Hyung-Chan An and Hoesook Yang. 2021. Fully Adaptive Stochastic Handling of Soft-Errors in Real-Time Systems. *IEEE Access* 9, 155058–155071. <https://doi.org/10.1109/ACCESS.2021.3128397>.
- [14] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [15] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [16] Marek Chrobak, Mathilde Hurand, and Jiří Sgall. 2009. Algorithms for Testing Fault-Tolerance of Sequenced Jobs. *Journal of Scheduling*. Author version. <https://iuuk.mff.cuni.cz/~sgall/ps/faults.pdf>.
- [17] Phebe Vayanos, Angelos Georgioui, and Han Yu. 2026. Robust Optimization with Decision-Dependent Information Discovery. *Management Science* 72, 2, 1509–1528. <https://doi.org/10.1287/mnsc.2021.00160>.
- [18] Eva Ley, Rebecca Marx, Maximilian Merkert, Tim Niemann, and Sebastian Stiller. 2026. Decision-dependent robust optimization with elementwise controllable uncertainty sets. *OR Spectrum*. <https://doi.org/10.1007/s00291-026-00856-z>.
- [19] PRISM-games developers. 2026. PRISM-games. Official tool documentation. Version 3.2.4 used in the experiments. <https://www.prismmodelchecker.org/games/>.
- [20] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.

- [21] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint* arXiv:1511.07163v1. <https://arxiv.org/abs/1511.07163>.
- [22] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [23] Namratha Gangamreddypalli, Constantin Enea, and Shaz Qadeer. 2026. Reduction for Structured Concurrent Programs. *arXiv preprint* arXiv:2601.13341. <https://arxiv.org/abs/2601.13341>.
- [24] Viacheslav Izosimov, Paul Pop, Petru Eles, and Zebo Peng. 2012. Scheduling and Optimization of Fault-Tolerant Embedded Systems with Transparency/Performance Trade-Offs. *ACM Transactions on Embedded Computing Systems* 11, 3, Article 61, 35 pages. <https://doi.org/10.1145/2345770.2345773>.
- [25] Bingham Wu, Wei Bao, Dong Yuan, and Bing Zhou. 2022. Competitive Analysis for Multi-Commodity Ski-Rental Problem. In *IJCAI*, 4672–4678. <https://doi.org/10.24963/ijcai.2022/648>.
- [26] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [27] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [28] OpenJDK. 2026. ConcurrentHashMap.java, jdk 17 u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [29] Ziwei Li, Bo Sun, Zhiqiu Zhang, Mohammad Hajiesmaili, Bingham Wu, Lin F. Yang, and Yang Gao. 2025. Combinatorial Ski Rental Problem: Robust and Learning-Augmented Algorithms. In *NeurIPS*. https://papers.nips.cc/paper_files/paper/2025/hash/ae3e173398d6e43fea63cbcc16fbaa98-Abstract-Conference.html.
- [30] Krzysztof Burdzy, Weining Kang, and Kavita Ramanan. 2009. The Skorokhod problem in a time-dependent interval. *Stochastic Processes and their Applications* 119, 2, 428–452. <https://doi.org/10.1016/j.spa.2008.03.001>.
- [31] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>.
- [32] Laurent Perron, Frédéric Didier, and Steven Gay. 2023. The CP-SAT-LP Solver (Invited Talk). In *CP, LIPIcs* 280, 3:1–3:2. <https://doi.org/10.4230/LIPIcs.CP.2023.3>.
- [33] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. [Online appendix; author code](#).
- [34] Dimitris Bertsimas and Melvyn Sim. 2003. Robust discrete optimization and network flows. *Mathematical Programming* 98, 49–71. <https://doi.org/10.1007/s10107-003-0396-4>.
- [35] Matthew Bold and Marc Goerigk. 2021. A compact reformulation of the two-stage robust resource-constrained project scheduling problem. *Computers & Operations Research*. <https://doi.org/10.1016/j.cor.2021.105232>. [Accepted manuscript](#).
- [36] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9. [Primary paper](#).
- [37] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [38] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Version 13 source-semantics and partial-repair supplement, commit 3e0dfbf. <https://github.com/research-artifact-archive/research-artifact/tree/3e0dfbf3e8ceaf5b8b8c904337333401629dae9>.